

PARTE 2 DE 2

IRIS Orquestrador: fazer o agente pensar barato.

Um chat que responde perguntas sobre uma fábrica inteira, consultando 14 MB de dados bagunçados, sem mandar 14 MB para o modelo. A tela existe. O agente não.

TEMPO ESTIMADO

3 a 4 dias
de trabalho

STACK

React, Express
e Responses API

PRÉ-REQUISITO

Node 20 e a
chave que eu te passo



Um agente com acesso à fábrica

Na parte 1 você consertou um app que já existia. Aqui você constrói um do zero, sobre uma base que já existe.

O objetivo é um chat onde alguém da operação pergunta em português sobre as máquinas da planta e recebe uma resposta correta, montada a partir dos dados reais. Isso é o coração do IRIS de verdade, reduzido ao osso: um modelo de linguagem que sozinho não sabe nada da sua fábrica, e um monte de dado que sozinho não responde pergunta nenhuma. Juntar os dois é o trabalho.

O QUE JÁ ESTÁ PRONTO

- A tela do chat em React, funcionando: envia mensagem, mostra estado de carregando, mostra erro.
- Uma API em Express com as rotas de dados da planta.
- Uma base com **129 ativos, 304 sensores, 190 mil leituras, 788 ordens de manutenção, 10 mil alarmes e 453 paradas de máquina**, que a API gera sozinha na primeira execução.

O QUE NÃO ESTÁ

- A rota `POST /api/chat` responde 501. É onde você trabalha.
- Não tem **uma linha sequer** de código da API do modelo neste repositório. Descobrir como aquilo funciona, lendo a documentação, faz parte do exercício.
- Nenhuma rota de dados filtra, agrega ou resume coisa alguma. Todas devolvem o conteúdo cru dos arquivos.

O backend é seu. Diferente da parte 1, aqui você pode criar rota, mudar estrutura e reescrever o que quiser dentro de `server/`. A única coisa que não se toca é o gerador da base: a bagunça precisa continuar sendo a mesma para todo mundo.

A chave e o resto

```
$ npm install
$ cp .env.example .env
$ npm run dev
```

Na primeira execução a API gera a base sozinha, o que leva alguns segundos e ocupa uns 14 MB em `server/dados/`. Depois disso sobe a API em `localhost:3001` e a tela em `localhost:5173`.

A chave da OpenAI eu te passo separado, junto com o nome do modelo. Ela vai no `.env`, que já está no `.gitignore`.

A CHAVE NUNCA CHEGA NO NAVEGADOR

Se ela aparecer no bundle do front, numa variável que começa com `VITE_`, ou numa requisição visível no DevTools, está errado, mesmo que funcione perfeitamente.

Pense em quem consegue abrir o DevTools de um site em produção: qualquer pessoa. Chave que o navegador vê é chave vazada, e quem paga a conta somos nós.

ANTES DE ESCREVER QUALQUER CÓDIGO

Abra estas rotas no navegador e olhe o tamanho da resposta na aba Network:

GET /api/estatisticas	contagens e bytes de cada coleção
GET /api/tudo	tudo de uma vez, menos as leituras
GET /api/leituras	todas as leituras da planta
GET /api/ativos	todos os ativos cadastrados

Esses quatro números vão definir todo o desenho da sua solução. Anote eles antes de começar.



Quatro frentes

01 Fazer o agente responder

POST /api/chat recebe as mensagens da conversa e precisa devolver a resposta do agente. Use a [Responses API da OpenAI](#), não a Chat Completions. Instale o que precisar.

O agente tem que dar conta das perguntas que aparecem na tela inicial. Nenhuma delas tem resposta pronta em lugar nenhum: todas exigem olhar dados e concluir alguma coisa.

02 Não queimar dinheiro

Token custa. O IRIS roda o dia inteiro, com várias pessoas perguntando ao mesmo tempo, e o que você desenhar aqui define a conta no fim do mês. Essa é a parte difícil, e está detalhada na página seguinte.

03 Botar ordem nos dados

A base saiu de três sistemas diferentes ao longo de seis anos e nunca foi padronizada. Antes do agente responder qualquer coisa com confiança, alguém precisa arrumar isso.

04 Uma UX que não irrita

A tela de hoje é o mínimo que funciona. Uma consulta ao modelo demora segundos, e a pessoa precisa perceber que algo está acontecendo. Precisa saber também [de onde veio a resposta](#): qual ativo, qual período, quantos registros o agente olhou. Resposta de agente sem procedência é chute com boa redação.

O problema de verdade

Um modelo de linguagem cobra por token, na entrada e na saída. Tudo que você manda para ele custa: a instrução, o histórico da conversa e, principalmente, os dados que você colocar na frente dele.

A planta desta base tem 190 mil leituras e 10 mil alarmes, e isso é um mês só. Uma fábrica de verdade acumula isso toda semana, para sempre. Se a sua solução depende de mandar os dados para o modelo peneirar, ela não sobrevive nem à primeira pergunta, e se sobrevivesse você não ia querer pagar.

O QUE EU QUERO VER NO RELATÓRIO

Meça. A resposta da API informa quantos tokens cada chamada consumiu. Mostre esse número na tela, em cada mensagem. Quem não mede não sabe.

Compare. Calcule quanto custaria mandar os dados crus e quanto custa a sua implementação. Escreva os dois números e a diferença entre eles.

Justifique. Diga por que o número onde você chegou é aceitável, e o que você faria se precisasse cortar ele pela metade de novo.

Não vou te dar um número alvo. Descobrir qual é a ordem de grandeza aceitável, e perceber o tamanho do abismo entre a solução ingênua e a boa, é justamente o que este exercício ensina.

O QUE VALE MAIS QUE QUALQUER ECONOMIA

Um agente que responde número errado, com confiança, é pior que um agente que não responde. A operação vai tomar decisão em cima do que ele falar. Se você tiver que escolher entre uma resposta barata e uma resposta certa, escolha a certa e me conte o preço.

Ninguém padronizou nada em seis anos

A base é real no sentido que importa: ela é bagunçada do jeito que dado de chão de fábrica é bagunçado. Três sistemas diferentes gravaram nela ao longo dos anos, cada um do seu jeito, e ninguém nunca parou para arrumar.

O QUE VOCÊ VAI ENCONTRAR, EM ALGUM LUGAR OU EM VÁRIOS

- o mesmo ativo cadastrado mais de uma vez, com o código escrito de formas diferentes
- campos com nomes diferentes para a mesma coisa, dependendo de quem gravou
- datas em mais de um formato, dentro do mesmo arquivo
- número gravado como texto, às vezes com vírgula decimal
- registros apontando para coisas que não existem
- valores impossíveis, de sensor com defeito
- vocabulário inconsistente em campos de status e severidade
- unidades de medida misturadas

Esse último item é o mais perigoso da lista, e vou deixar você descobrir por quê.

UMA DICA SÓ, E É A ÚNICA

Antes de escrever qualquer linha do agente, pense em como seria a resposta ideal para "como está a Prensa 01 hoje". Escreva ela à mão, do jeito que você gostaria de ler. Agora conte quantos dados uma pessoa precisaria olhar para escrever aquilo, e compare com o tamanho do que a API te devolve hoje. A diferença entre esses dois tamanhos é o desafio inteiro.

POR QUE ESTE EXERCÍCIO EXISTE

Fazer um modelo responder é fácil, tem tutorial de quinze minutos. Fazer ele responder certo, sobre dados que ninguém organizou, sem torrar orçamento, é o que a gente faz aqui todo dia.

Este exercício é uma versão pequena e sem cliente esperando de um problema que está em produção agora. As decisões que você vai tomar nele são as mesmas que a gente tomou no IRIS, e algumas a gente tomou errado da primeira vez.

O QUE ME MANDAR

O repositório funcionando, um RELATORIO.md com o desenho da sua solução, o que você achou de errado nos dados e os números de token, e uns prints ou um vídeo curto do chat respondendo três perguntas diferentes, para eu ver funcionando sem precisar da chave.

Travou por mais de uma hora no mesmo ponto, me chama. Levar quatro dias é normal. Levar quatro dias travado no mesmo erro, não.

